TP3 — Perceptrón Simple y Multicapa

Sistemas de Inteligencia Artificial — ITBA 2026

Equipo SIA

Marco teórico y objetivos

Objetivos del TP3

- **Ejercicio 1**: Estudio de fraude online con perceptrón simple lineal vs no lineal (knowledge distillation de *BigModel*).
- **Ejercicio 2**: Clasificación de dígitos con MLP, variando tasa de aprendizaje, arquitectura y mecanismo de optimización.
- **Ejercicio 3**: Mejorar el clasificador con `more_digits.csv` buscando **accuracy** $\geq 98\,\%$.

Esta presentación resume las decisiones más relevantes de **EJ1, EJ2 y EJ3**.

Arquitectura del MLP (Clase 11, Clase 13 - Backprop)

Forward: $V_j^m = \theta\left(\sum_k V_k^{m-1} \cdot w_{jk}^m\right)$ con $V^0 = x$, $O = V^M$.

Loss (MSE): $E(w) = \frac{1}{2N} \sum_{\mu=1}^{N} \sum_i \left(\zeta_i^\mu - O_i^\mu\right)^2$

Backward (regla de la cadena):

$$\delta^{(L)} = (O - t) \odot f'(h^{(L)}), \quad \delta^{(m)} = \left(W^{(m+1)\top} \delta^{(m+1)}\right) \odot f'(h^{(m)})$$

$$\frac{\partial E}{\partial W^{(m)}} = \frac{1}{N} \delta^{(m)} V^{(m-1)\top}, \quad w^{\text{nuevo}} = w^{\text{ant}} - \eta \frac{\partial E}{\partial w}$$

Capacidad y generalización (Clase 13 - Regularización): más capacidad $\Rightarrow$ más riesgo de overfitting; brecha train$-$validación crece con la capacidad.

EJ1 — Prevención de fraude

EJ1 — Contexto y setup

Problema: CompanyX usa un modelo grande y caro, **BigModel**, para estimar la probabilidad de fraude en transacciones online.

Objetivo: construir un modelo más chico, **TinyModel**, que aproxime la salida de BigModel con menor costo de inferencia.

Qué se pide estudiar:

- Comparar un perceptrón simple lineal contra uno no lineal.
- Elegir el modelo con mejor capacidad de aprendizaje para generalización.
- Recomendar un umbral de detección de fraude.

Criterio de negocio: priorizar la reducción de falsos negativos, porque un fraude real no detectado es más costoso que revisar una operación sospechosa.

EJ1 — Normalización con z-score

Usamos **z-score** para llevar las variables a una escala comparable antes del entrenamiento.

$$z = \frac{x - \mu}{\sigma}$$

donde μ es la **media** y σ es el **desvío estándar**.

Ejemplo real sobre `amount_usd` del dataset:

$$x = [42{,}07]\,, \qquad \mu = [109{,}07]\,, \qquad \sigma = [157{,}74]$$

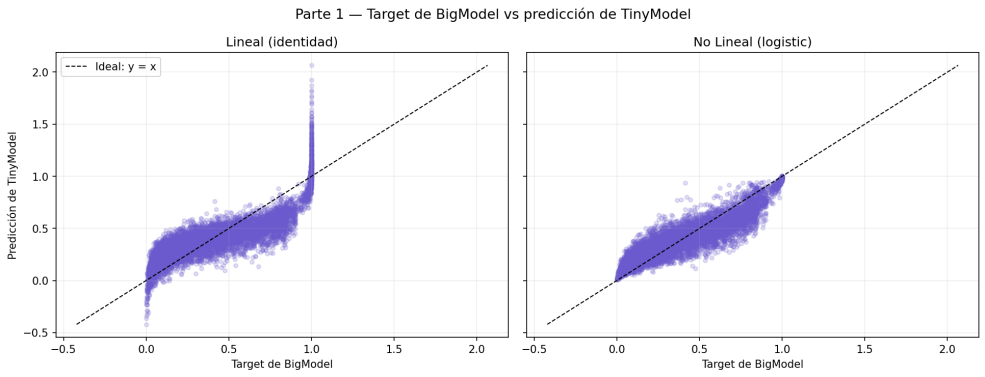
$$z = \left[\frac{42{,}07 - 109{,}07}{157{,}74}\right] = [-0{,}425]$$

Entrenamiento

EJ1 — Parámetros de la Parte 1

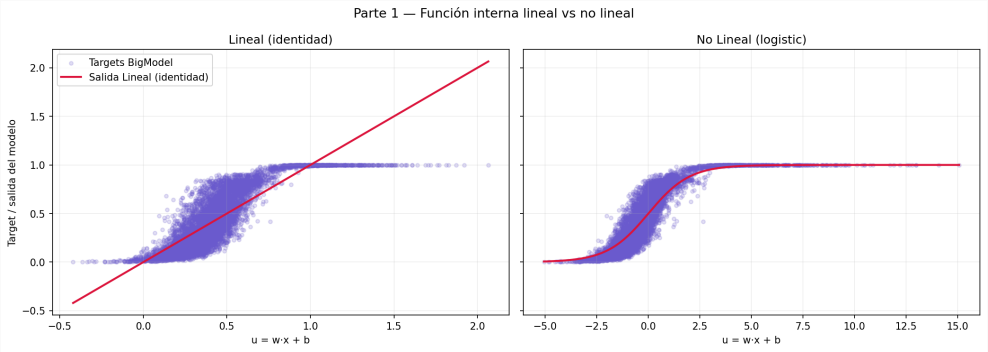
Parámetro	Valor
Modelos comparados	lineal (identity) vs no lineal (logistic)
Activaciones	identity, logistic ($\beta = 1{,}0$)
Optimización	GD
Learning rate	10^{-3}
Batch size	32
Épocas	500
Normalización	z-score
Semillas	$0, 1, 2, 3, 4$

EJ1 — Target vs predicción



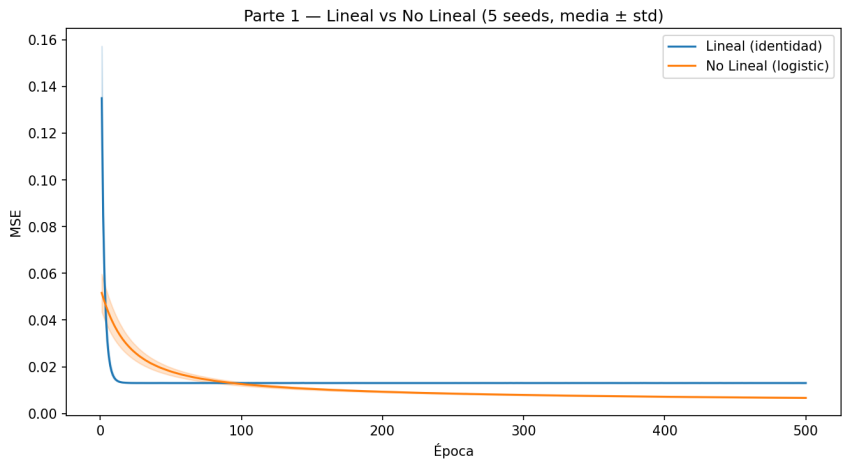
Se comparan las activaciones `identity` y `logistic`.

EJ1 — Función interna



Se comparan las funciones de activación `identity` y `logistic`. El perceptrón no lineal mantiene la salida acotada en $[0, 1]$.

EJ1 — Error MSE



El perceptrón no lineal reduce el **MSE en 48.9 %** respecto del lineal.

EJ1 — Conclusiones Parte 1

- El perceptrón **lineal** muestra **underfitting**: no logra aproximar bien la salida de BigModel.
- El modelo lineal **satura su capacidad** rápidamente y converge a un error más alto.
- El perceptrón **no lineal** captura mejor la relación entrada-salida y mantiene la predicción en el rango $[0, 1]$.
- En este experimento, el no lineal logra un **MSE significativamente menor** sin mostrar una saturación comparable.
- Para generalización, se selecciona el **perceptrón simple no lineal**.

Generalización

EJ1 — Setup experimental de generalización

Parámetro	Valor
Manejo de datos	5-Fold
Modelo	perceptrón simple no lineal
Activación	logística
Optimizador base	GD
Batch size base	32
Learning rate base	10^{-3}
Épocas máximas	500
Early stopping	sí
Semillas	$0, 1, 2, 3, 4$

A partir de esta configuración base, se exploran **batch size**, **learning rate** y **optimizador**.

EJ1 — Métricas de evaluación

Como criterio de diseño, queremos **minimizar los falsos negativos (FN)**: casos donde una operación fraudulenta se clasifica como no fraudulenta. Aun así, también contemplamos el resto de los errores.

- **F2**

$$F_\beta = (1+\beta^2)\cdot\frac{\text{Precision}\,\text{Recall}}{\beta^2\,\text{Precision}+\text{Recall}} \qquad \Rightarrow \qquad F_2 = 5\cdot\frac{\text{Precision}\,\text{Recall}}{4\,\text{Precision}+\text{Recall}}$$

- **MSE**

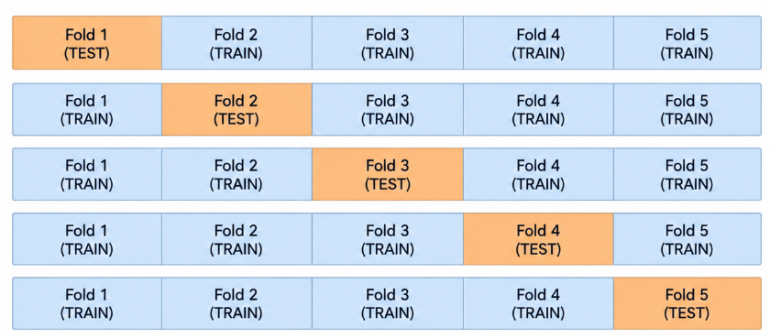
$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2$$

  donde y_i es la salida de **BigModel** y $\hat{y}_i$ es la predicción de nuestro modelo.

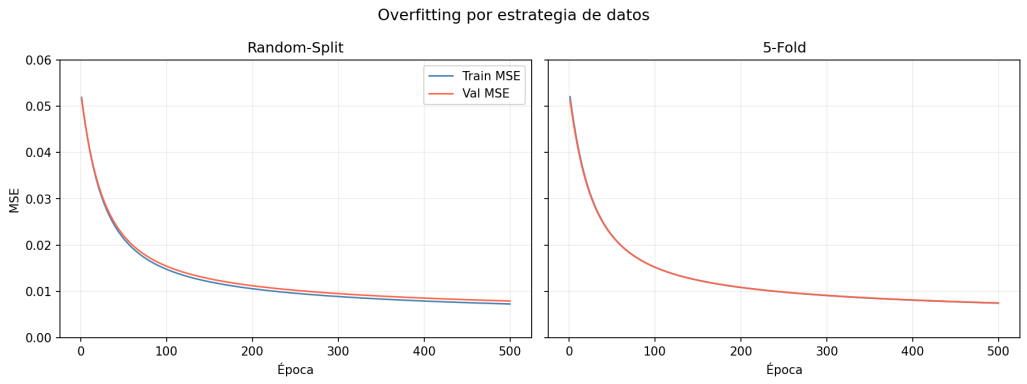
- **Costo** $= 2\,FN + FP$

EJ1 — Estrategias de datos

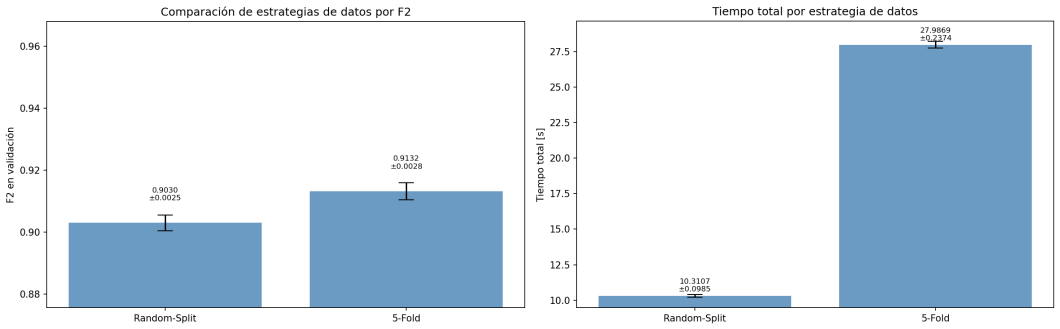
- **Random-Split**: partición aleatoria train/validación.
- **5-Fold**: divide el dataset en folds y rota cuál queda para validación.



EJ1 — Curvas train/val

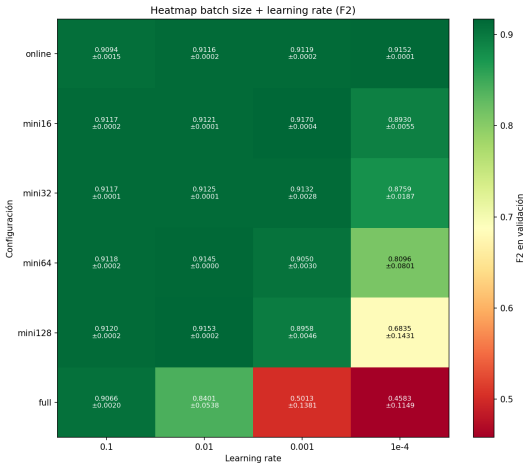


EJ1 — Estrategias de datos

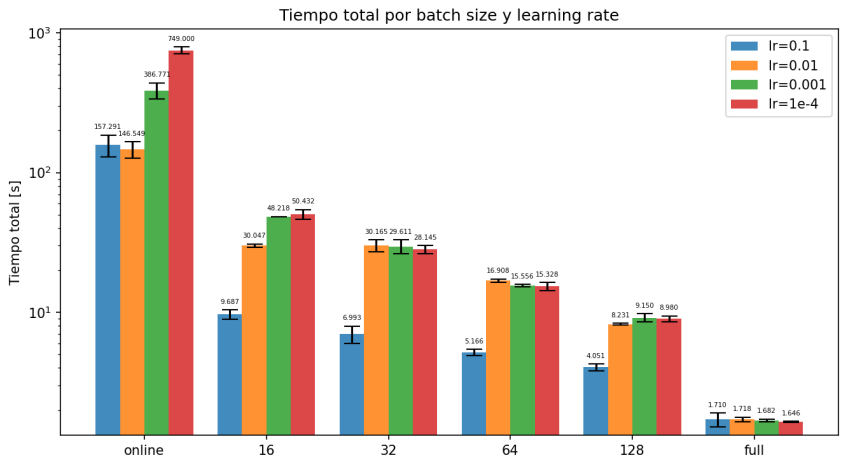


Se selecciona **S3**.

EJ1 — Batch size y learning rate

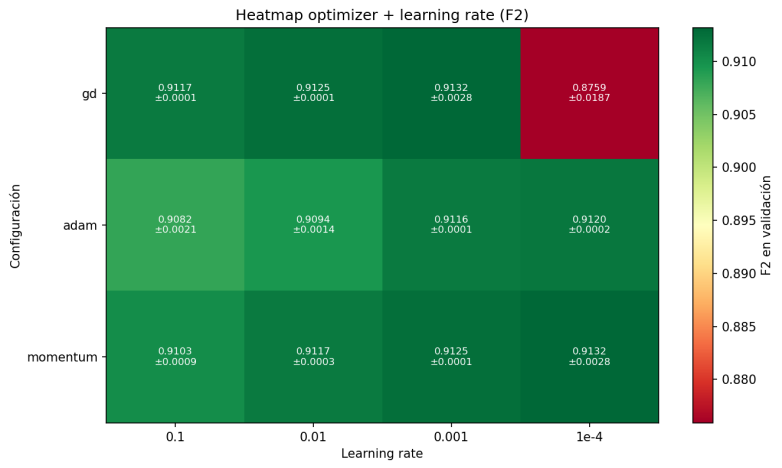


EJ1 — Tiempo por batch size y learning rate

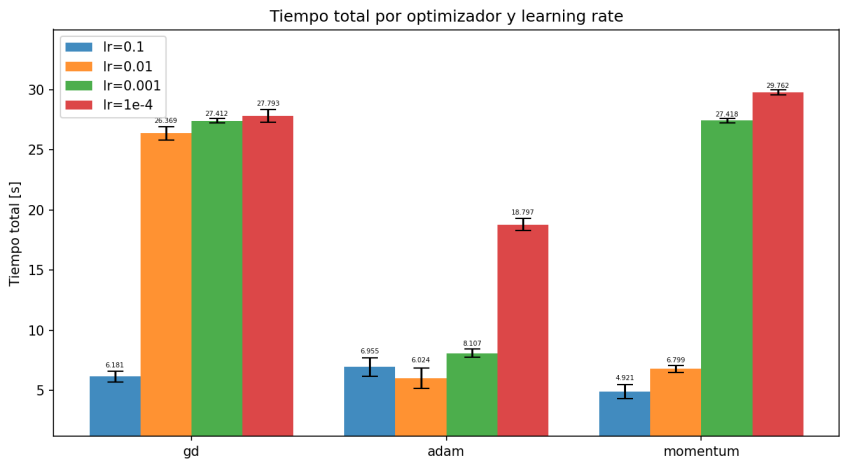


Se elige **batch_size = 16**.

EJ1 — Screening de optimizadores

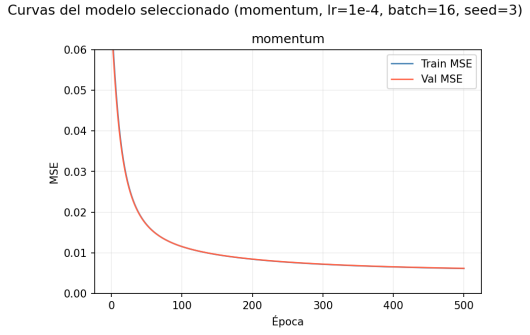


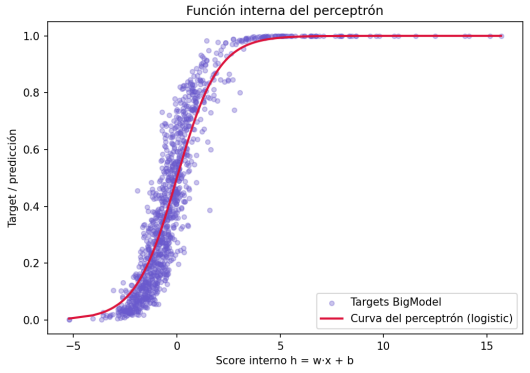
EJ1 — Tiempo por optimizador



Se elige **Momentum** ($\alpha = 0{,}9$).

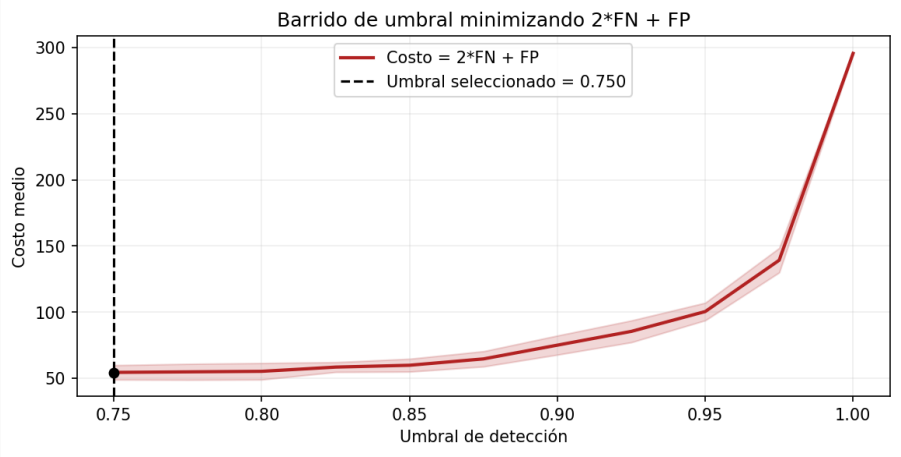
EJ1 — Modelo seleccionado: aprendizaje e interpretación





Modelo: **Momentum ($\alpha = 0{,}9$), lr $= 10^{-4}$, batch = 16** y **S3**.

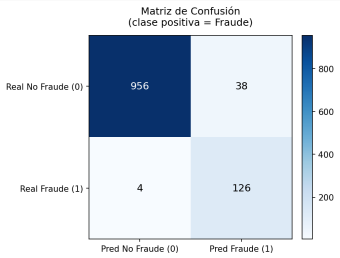
EJ1 — Selección de umbral



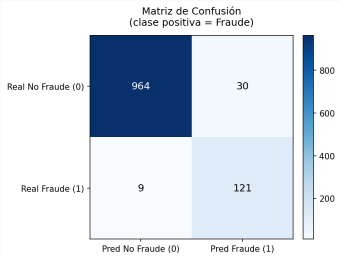
Se selecciona umbral **0.75**.

EJ1 — Matrices de confusión por umbral

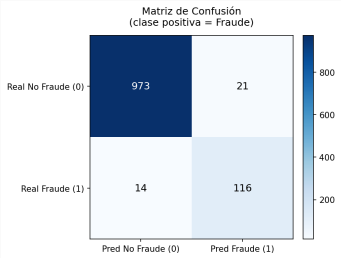
$t = 0{,}75$



$t = 0{,}775$



$t = 0{,}8$



Se selecciona **0.75**, dejando solo **4** casos de fraude real sin detectar.

EJ1 — Modelo final para el cliente

- Estrategia de datos: **S3 (5-Fold CV)**
- Modelo: **perceptrón simple no lineal** con activación **logística**
- Optimizador: **Momentum** ($\alpha = 0{,}9$)
- Learning rate: 10^{-4}
- Batch size: **16**
- Umbral recomendado: **0.75**

- **Precision**: 0,7683
- **Recall**: 0,9692
- **F1**: 0,8571
- **F2**: 0,9211
- **MSE**: 0,00590
- **Costo** ($2FN + FP$): 46

EJ2 — Clasificación de dígitos

Contexto del ejercicio

Problema: CompanyX necesita un sistema automático para clasificar dígitos escritos a mano (0–9) a partir de imágenes de 28×28 pixels.

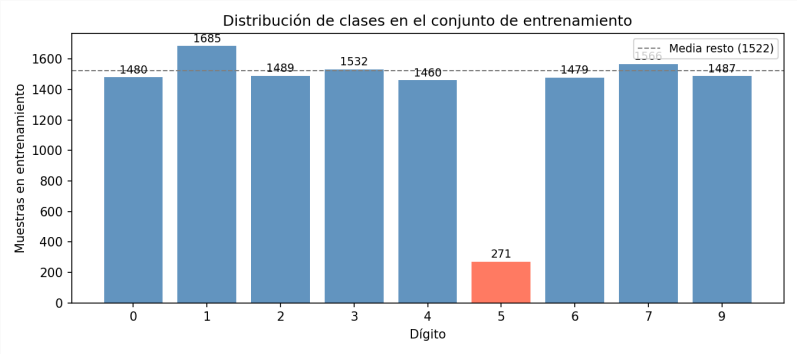
Modelo: Perceptrón Multicapa (MLP) con salida de 10 neuronas (una por dígito), entrenado con MSE.

Estrategia experimental:

1. Ver dependencia entre batch size y learning rate
2. Ver variación de accuracy según optimizador y learning rate
3. Explorar arquitecturas: ¿más capas o más nodos por capa?
4. Evaluar funciones de activación en capas ocultas
5. Analizar el mejor modelo sobre el conjunto de test

División: 80 % train / 20 % val (estratificado) sobre `digits.csv`; `digits_test.csv` como "mundo real".

Análisis del dataset — Distribución de clases



Distribución de muestras por dígito en el conjunto de entrenamiento. Se observa una marcada subrepresentación del 5 y la ausencia total del 8.

Análisis del dataset — Observaciones e hipótesis

Dataset / clase	0	1	2	3	4	5	6	7	8	9
Train (12 449)	1480	1685	1489	1532	1460	**271**	1479	1566	**0**	1487
Test (2 497)	245	283	258	252	245	223	239	257	**243**	252

Observaciones:

- El train **no contiene la clase 8** — el modelo nunca verá un ejemplo de ese dígito.
- La clase 5 tiene solo **271 muestras** ($\sim$2 % del train) contra $\sim$12 % para el resto.
- El test sí incluye 243 muestras del 8 ($\sim$10 % del test).

Hipótesis:

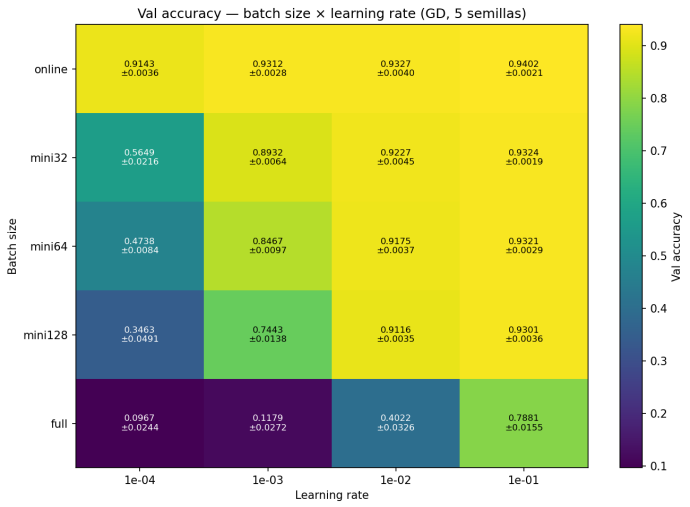
- El 8 será sistemáticamente mal clasificado (techo de test accuracy $\approx$ 90 %).
- El dígito 5 tendrá menor F1 por la escasez de muestras.

Exp1 — Parámetros iniciales

Parámetro	Valor
Arquitectura	[784, 40, 20, 10]
Activación oculta	tanh
Activación de salida	logistic
β (pendiente sigmoidea)	1.0
Inicialización	random_normal, init_scale=0.1
Optimizador	GD (gradiente descendente)
Épocas máximas	500
Función de pérdida	MSE
Early stopping	patience=50, $\varepsilon = 0{,}001$
Seeds	[0, 1, 2, 3, 4]

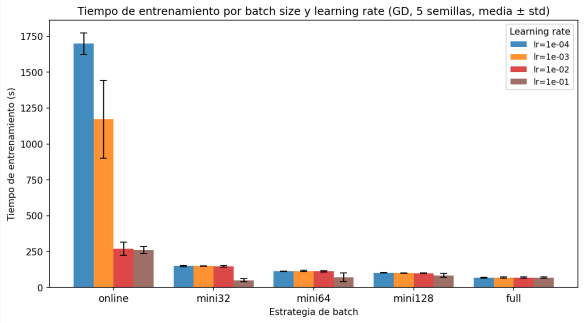
Valores por defecto; se varían batch size y learning rate en este experimento.

Exp1 — Batch size × learning rate



5 batch sizes × 5 learning rates × 5 seeds. La zona óptima se concentra en lr $\in [10^{-3}, 5{\cdot}10^{-3}]$ con batch $\in [8, 128]$.

Exp1 — ¿Online o mini-batch?



Hallazgos:

- El entrenamiento **online** (batch=1) logra la mejor val accuracy, pero es $\sim 10\times$ más lento.
- Mini-batch 32 y 64 obtienen resultados comparables con mucho menor tiempo de entrenamiento.
- El learning rate adecuado **depende del batch size**: batch grande necesita lr más alto; online necesita lr más bajo.

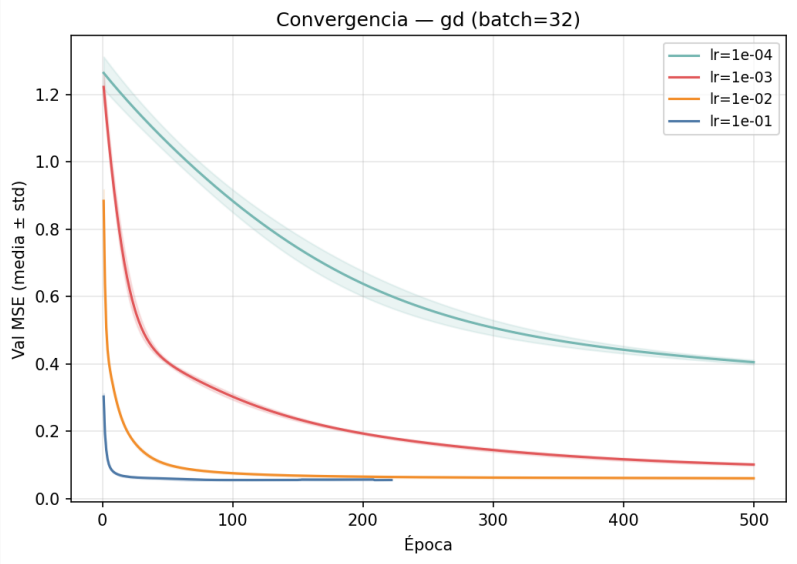
Decisión: batch size = **32**. Buen balance entre velocidad y accuracy; además permite flexibilidad en la elección de lr.

Exp2 — Parámetros iniciales

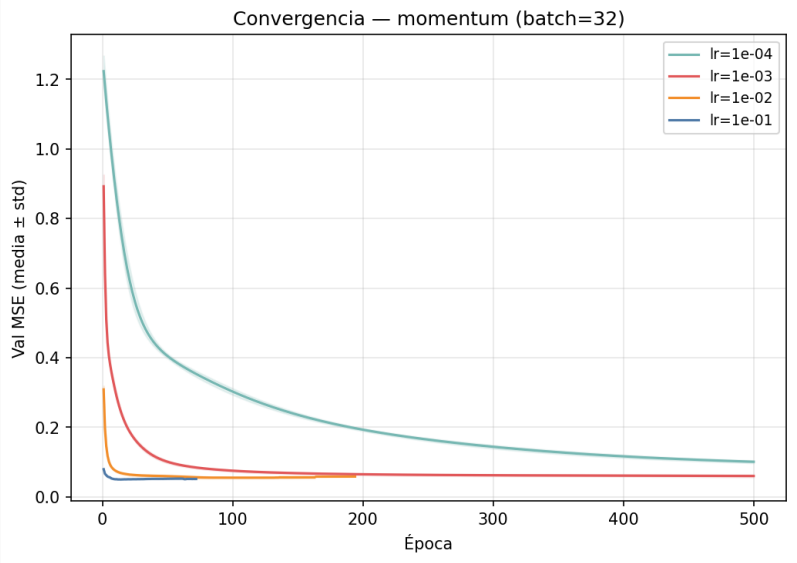
Parámetro	Valor
Arquitectura	[784, 40, 20, 10]
Activación oculta	tanh
Activación de salida	logistic
β (pendiente sigmoidea)	1.0
Inicialización	random_normal, init_scale=0.1
Batch size	32
Épocas máximas	500
Función de pérdida	MSE
Early stopping	patience=50, $\varepsilon = 0{,}001$
Seeds	[0, 1, 2, 3, 4]

Se varían optimizador y learning rate.

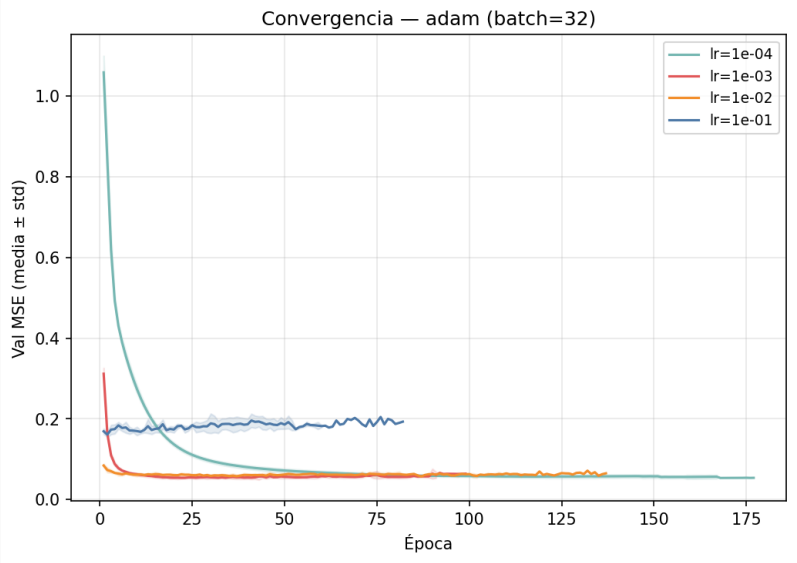
Exp2 — Convergencia: GD



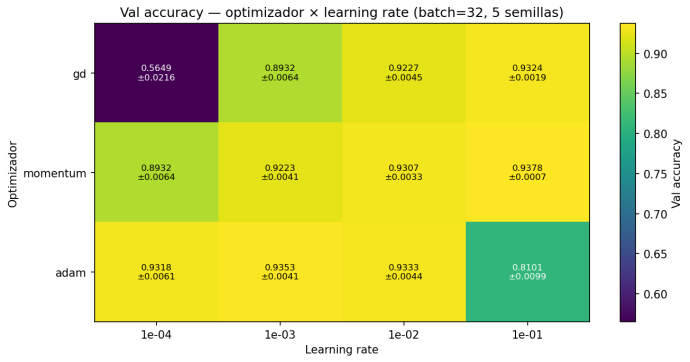
Exp2 — Convergencia: Momentum ($\alpha = 0{,}9$)



Exp2 — Convergencia: Adam (β_1=0,9, β_2=0,999, ε=10^{-8})



Exp2 — Cada optimizador tiene su learning rate óptimo



3 optimizadores $\times$ 4 learning rates $\times$ 5 seeds. GD necesita lr alto ($\sim$0.1), Momentum funciona en rango intermedio, Adam rinde mejor con lr $\sim 10^{-3}$.

Exp2 — ¿Por qué Adam con lr $= 10^{-3}$?

- **Adam** adapta la tasa de aprendizaje por parámetro (momentos de primer y segundo orden), por lo que trabaja bien con lr bajos y converge rápido.
- **GD** no tiene adaptación: necesita lr alto para moverse, pero eso genera inestabilidad.
- **Momentum** queda en un punto intermedio: acelera en direcciones consistentes pero no adapta por parámetro.
- Conclusión: **Adam con lr** $= 10^{-3}$ como optimizador base para los experimentos siguientes.

Exp3 — ¿Más capas o más nodos por capa?

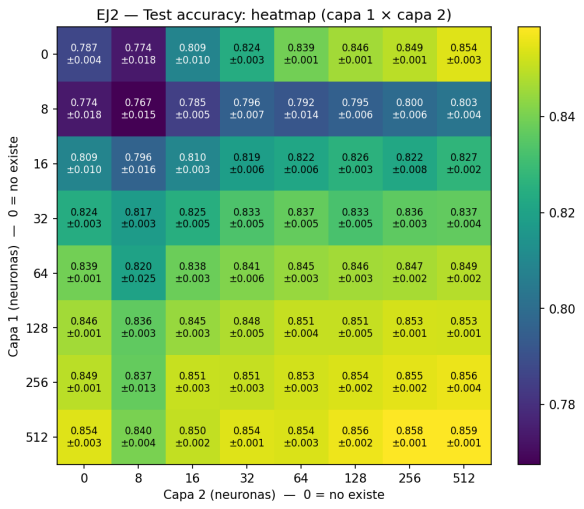
Pregunta: dada una arquitectura de hasta 2 capas ocultas, ¿cómo conviene distribuir los nodos entre la primera y la segunda capa? ¿Conviene una sola capa ancha o repartir en dos capas?

Diseño: barrido 2D de $n_1 \times n_2$ con $n_1, n_2 \in \{0, 8, 16, 32, 64, 128, 256, 512\}$. Convención: $n = 0$ significa que esa capa *no existe* (los pares $(0, n)$ y $(n, 0)$ son el mismo MLP $[784, n, 10]$, así que se computan una sola vez).

- Total: **57 configuraciones únicas** $\times$ 5 seeds = 285 runs.
- Tres clases de arquitectura cubiertas en una sola grilla: *regresión logística* (sin capas ocultas), *1 capa oculta* de tamaño variable, y *2 capas ocultas* con todas las combinaciones.

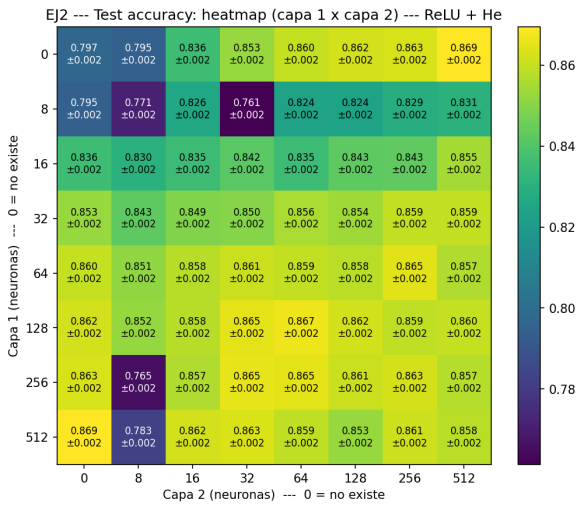
Configuración: Adam lr= 10^{-3}, batch=32, tanh, patience=50, max_epochs=500, init scale=0.1.

Exp3 — Heatmap de test accuracy con tanh



5 seeds por celda. Cada cell: mean $\pm$ std. Ganador con tanh: $(n_1, n_2) = (512, 512)$, test $= 0{,}8587 \pm 0{,}001$.

Exp3 — Heatmap de test accuracy con ReLU

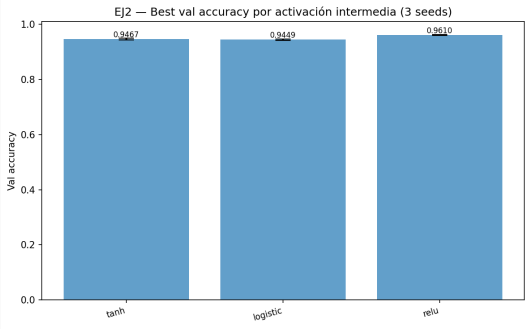


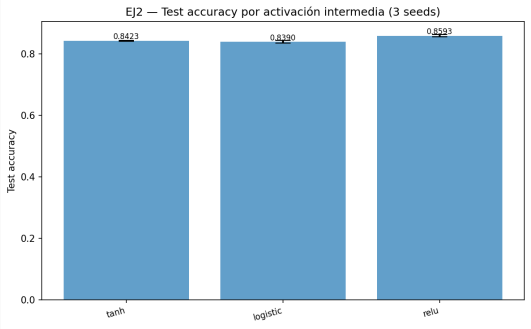
1 seed por celda (std fijo $\pm 0{,}002$ para visualización). Configuración idéntica al heatmap tanh excepto activación + init He.

Exp3 — Lectura comparada (tanh vs ReLU)

- **Con tanh**: el ganador era $[784, 512, 512, 10]$ con test $\sim$0.859. Wider $\succ$ deeper, capacidad ayuda hasta el extremo del barrido.
- **Con ReLU**: el ganador es $[784, 512, 10]$ (**1 sola capa oculta**) con test $\approx$ 0,869. Agregar una segunda capa NO mejora: $(512, 512)$ baja a 0,858.
- **Implicancia clave**: el "ganador" del barrido depende de la activación. La conclusión "más grande es mejor" es *específica de tanh*; con ReLU una sola capa ancha alcanza.
- **Decisión final**: para el modelo seleccionado optamos por $[784, 512, 10]$ + ReLU + He init (mejor test del barrido, arquitectura más simple).

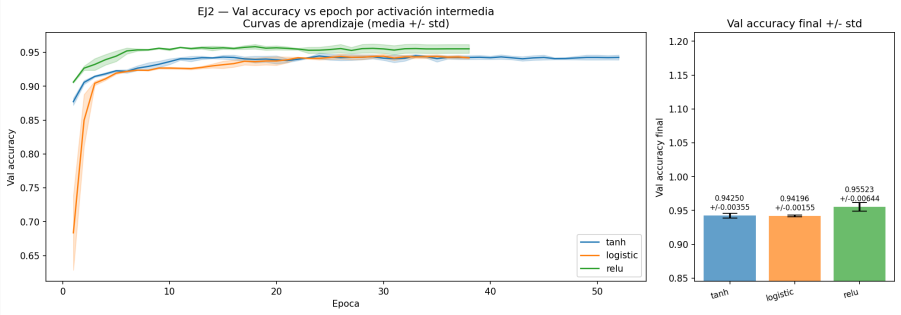
Exp4 — Activaciones en capas ocultas





Arquitecturas [784, 256, 10] y [784, 128, 64, 10], Adam lr= 10^{-3}, batch=32. Tres funciones de activación evaluadas: **logistic**, **tanh** y **ReLU**.

Exp4 — Convergencia por activación



ReLU converge más rápido y alcanza la mejor val accuracy. Logistic sufre de saturación (derivada pequeña) y converge más lento.

Exp4 — Conclusión: elección de activación

Activación	Arquitectura	Val accuracy	Test accuracy
ReLU	[784, 128, 64, 10]	$0{,}9616 \pm 0{,}003$	$0{,}8640 \pm 0{,}002$
Tanh	[784, 128, 64, 10]	$0{,}9497 \pm 0{,}003$	$0{,}8515 \pm 0{,}002$
Logistic	[784, 128, 64, 10]	$0{,}9485 \pm 0{,}004$	$0{,}8467 \pm 0{,}002$

- Las tres funciones de activación dan resultados aceptables.
- **ReLU** es superior: no satura para valores positivos y propaga gradientes más eficazmente.
- **Logistic** es la más lenta en convergencia (derivada pequeña en saturación).
- Decisión: usamos **ReLU** como activación por defecto para los experimentos siguientes.

Exp5 — Configuración del mejor modelo

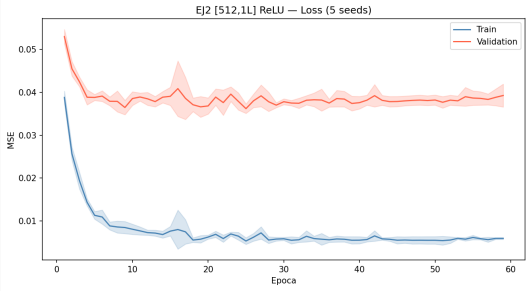
- **Arquitectura**: **[784, 512, 10]** (1 capa oculta, 407 050 parámetros)

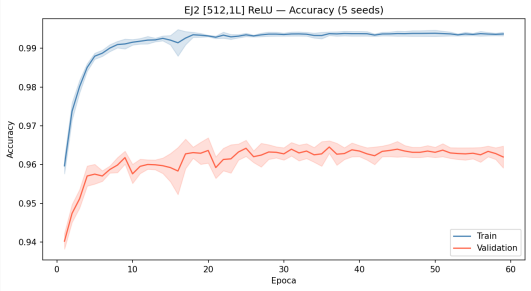
  Ganadora del heatmap ReLU del Exp3.
- **Optimizador**: Adam ($\eta = 10^{-3}$, $\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 10^{-8}$)
- **Batch size**: 32, init He normal ($\sigma = \sqrt{2/n_{\text{in}}}$)
- **Activación oculta**: **ReLU** (ganadora del Exp4)
- 5 seeds, hasta 150 epochs con early stopping (patience=20)

Resultados (5 seeds, mean $\pm$ std):

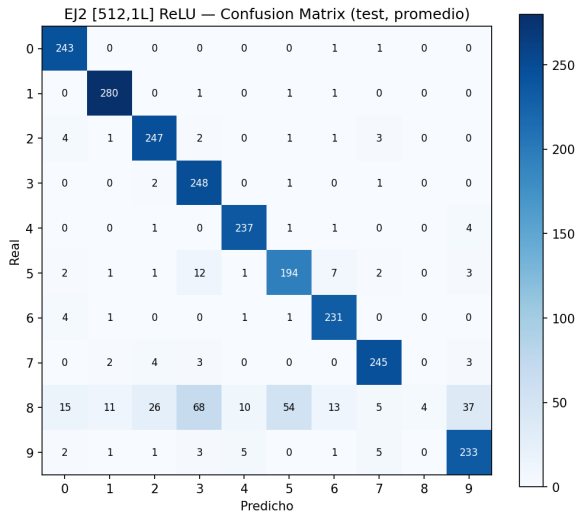
- Val accuracy: $0{,}9661 \pm 0{,}0020$
- Test accuracy: $0{,}8659 \pm 0{,}0046$
- Macro F1: $0{,}8215 \pm 0{,}0083$

Exp5 — Curvas de aprendizaje (5 seeds)



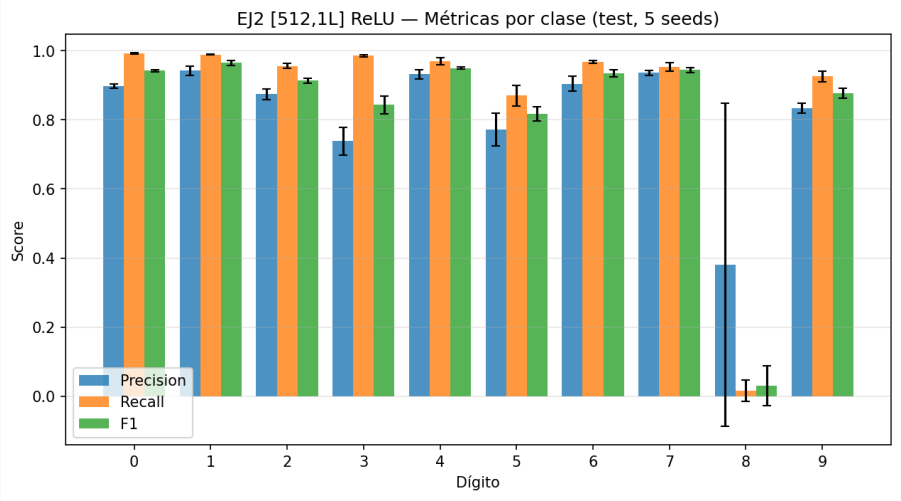


Exp5 — Matriz de confusión (test, promedio 5 seeds)



La fila del 8 es toda incorrecta: el modelo nunca vio un 8. Se confunde principalmente con 5 (23 %) y 3 (22 %).

Exp5 — Métricas por clase (test)



F1 del dígito 8 = 0.000 (ausente en train). El dígito 5 (271 muestras) también tiene F1 bajo ($\sim$0.75). Demás dígitos: F1 $\geq$ 0.84.

Exp5 — Análisis de confusiones por dígito (test)

Real	Acc clase	Top-1 confusión	Top-2 confusión
0	0.992	→6: 0.4 %	→7: 0.4 %
1	0.989	→3: 0.4 %	→5: 0.4 %
2	0.954	→0: 1.5 %	→7: 1.2 %
3	0.984	→2: 0.8 %	→5: 0.4 %
4	0.971	→9: 1.6 %	→2: 0.4 %
5	**0.870**	**→3: 5.4 %**	**→6: 3.1 %**
6	0.971	→0: 1.7 %	→1: 0.4 %
7	0.953	→2: 1.6 %	→3: 1.2 %
8	**0.016**	**→3: 28.0 %**	**→5: 22.2 %**
9	0.928	→4: 2.0 %	→7: 2.0 %

El 8, ausente del train, se reparte entre 3 (28 %) y 5 (22 %) — el modelo "los manda a cualquier lado". El 5, con solo 271 muestras en train, sigue siendo la clase peor aprendida.

EJ3 — Mejor accuracy con `more_digits.csv`

Setup del EJ3

Cambio principal: el train pasa a ser `more_digits.csv` ($\sim$15.7k muestras, **10 clases incluyendo el 8**).

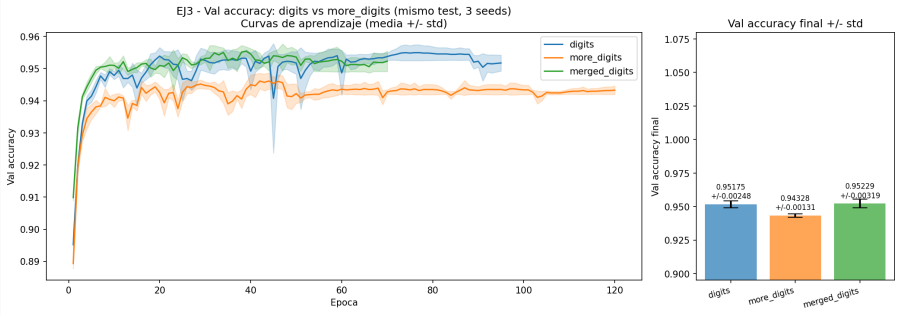
Test: el mismo `digits_test.csv` del EJ2 (mundo real) $\Rightarrow$ comparación directa con EJ2.

Objetivo: accuracy $\geq 98\,\%$ en test.

Técnicas adoptadas:

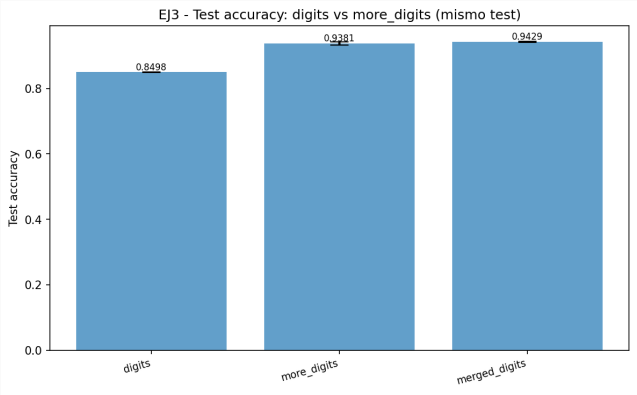
- Arquitectura más grande: $[784, 256, 128, 10]$ (235,146 params).
- Más epochs (150) + patience (20).
- Weight decay liviano ($\lambda = 10^{-5}$) — regularización L2 (Clase 13 - Regularización).
- Init scale más chico (0.05) para mejor estabilidad.
- Batch size 64 (más grande $\Rightarrow$ menos varianza por step).

Comparación de datasets (*aislando el efecto de los datos*)



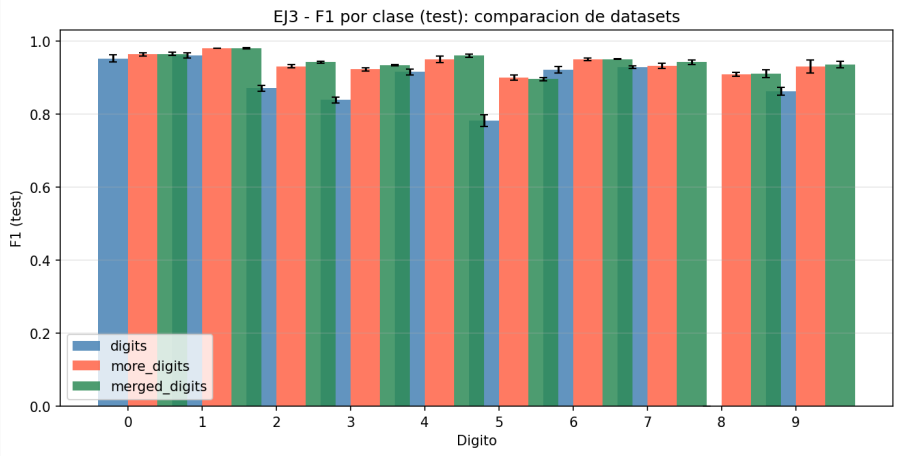
Misma arquitectura $[784, 128, 64, 10]$, mismo test, sólo cambia el train.

Test accuracy: digits vs more_digits



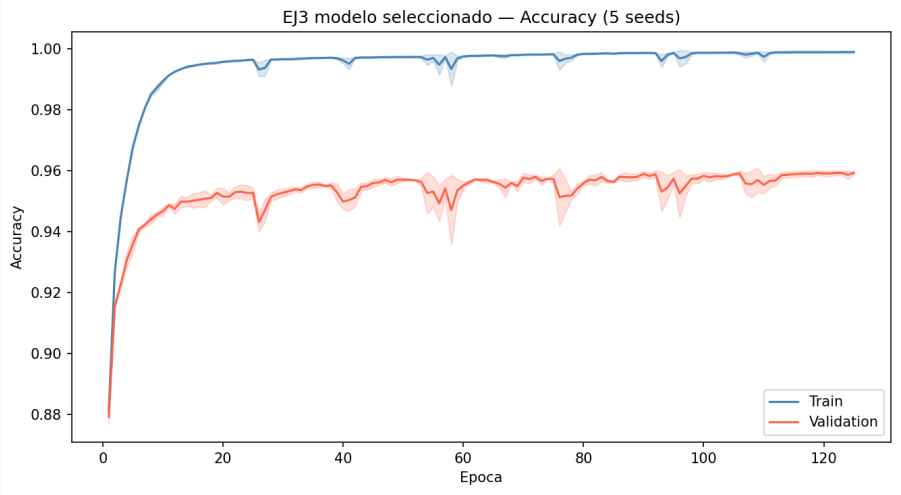
Salto de $\sim$80 % a $>$95 % sólo por incluir la clase 8 en train y tener $\sim$26 % más datos.

F1 por clase (test): digits vs more_digits

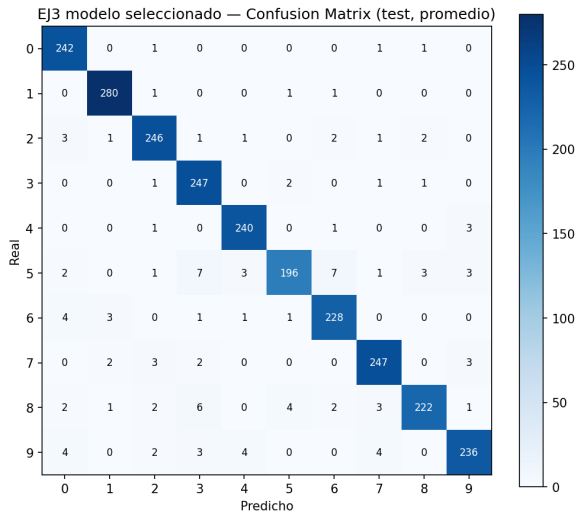


El dígito 8 pasa de F1=0 (no se entrena) a $\sim$0.95.

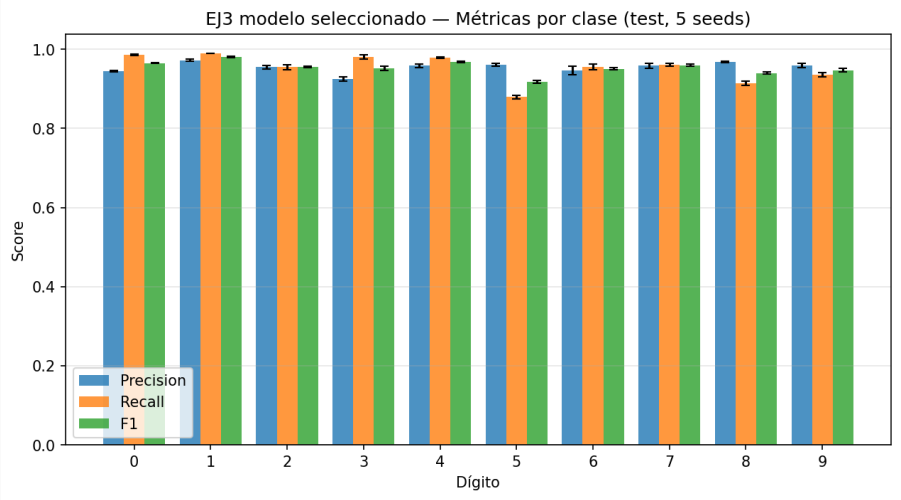
EJ3 modelo seleccionado — accuracy curves



EJ3 — Confusion matrix (test, promedio sobre 5 seeds)



EJ3 — Métricas por clase



EJ3 — Análisis de confusiones por dígito (test)

Real	Acc clase	Top-1 confusión	Top-2 confusión
0	0.988	→2: 0.4 %	→7: 0.4 %
1	0.989	→2: 0.4 %	→5: 0.4 %
2	0.957	→0: 1.2 %	→6: 0.8 %
3	0.980	→5: 0.8 %	→2: 0.4 %
4	0.980	→9: 1.2 %	→2: 0.4 %
5	0.879	→3: 3.1 %	→6: 3.1 %
6	0.958	→0: 1.7 %	→1: 1.3 %
7	0.961	→2: 1.2 %	→9: 1.2 %
8	**0.914**	**→3: 2.5 %**	**→5: 1.6 %**
9	0.933	→0: 1.6 %	→4: 1.6 %

El 8 pasa de acc=0.000 (EJ2) a 0.914 (EJ3) — confirma el efecto de incluirlo en el train. Las confusiones residuales del 5 y 8 son con 3, 5, 6 — patrón consistente con similitud visual.

EJ3 — Resultados finales y discusión del 98 %

Mejor resultado obtenido (3 seeds):

- Test accuracy: $0{,}9546 \pm 0{,}0007$ (95.5 %)
- Val accuracy: $0{,}9603 \pm 0{,}0001$
- Macro F1: 0,9536 (vs 0,7942 en EJ2)
- F1 mínimo (dígito 5): 0,918 (vs 0,749 en EJ2)

No alcanzamos 98 % en test. Lo que obtuvimos:

- Train acc $\rightarrow$ 0.999 con val acc plateau en $\sim$0.96 (overfitting clásico, Clase 13 - Regularización).
- El gap train$-$val es de $\sim$0.04, indicando que la red está saturando en su capacidad de generalización con este pipeline.

Para llegar a 98 % se necesitarían: data augmentation (rotaciones, traslaciones — Clase 13), arquitecturas convolucionales, o ensembling. Quedó fuera del scope de un MLP básico.

¿Qué es "feature engineering"?

Dada una imagen de 28×28, lo más natural es pasarle al MLP el *vector aplanado de 784 pixels*. Pero el MLP tiene que aprender DESDE CERO toda estructura útil de la imagen.

Feature engineering: pre-calcular "descriptores" de la imagen y dárselos al MLP *además* de los pixels. La red ya no arranca desde cero — empieza con info ya destilada.

Por qué puede ayudar:

- Algunos descriptores son *difíciles* de aprender por un MLP (ej. "cuántos huecos tiene el dígito" requiere análisis no-local que un MLP de 2 capas no captura bien).
- Acelera la convergencia: la red usa los pixels para los detalles y los features para la macro-estructura.

Inputs del modelo final: $784 + 56 + 7 + 5 + 324 = 1176$ features.

Feature 1 — Projection features (proj, 56)

Idea: sumar los pixels por fila y por columna.

$$\text{row}_i = \sum_{j=0}^{27} \text{img}[i, j], \qquad \text{col}_j = \sum_{i=0}^{27} \text{img}[i, j]$$

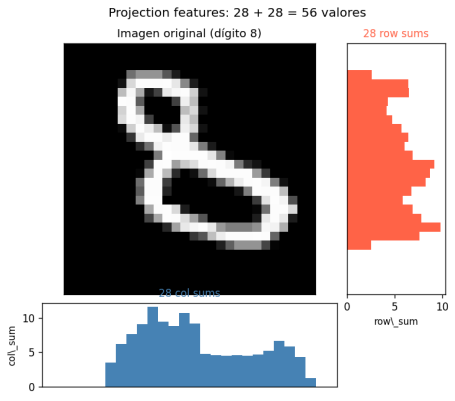
Total: $28 + 28 = 56$ features.

Qué captura: la *silueta* del dígito.

- Un **1** concentra masa en el centro.
- Un **0** tiene masa distribuida en todas las columnas.
- Un **4** tiene una col vertical fuerte y una horizontal en el medio.

Lectura técnica: es la base más simple del OCR pre-deep-learning. Funciona pero aporta poco a un MLP que ya tiene los pixels (puede inferirlas implícitamente en la primera capa).

Projection — ejemplo visual sobre un 8



Row sums (derecha): tinta vertical, dos picos en los bordes del 8 + valle en el medio. Col sums (abajo): tinta a izquierda y derecha, mínimo en el centro.

Feature 2 — Hu moments (hu, 7)

Idea: 7 invariantes estadísticos de la imagen, derivados de los *momentos centrales* normalizados.

Cómo se construyen (Hu, 1962):

1. Calcular momentos $m_{pq} = \sum_{x,y} x^p y^q \,\mathrm{img}(x, y)$.
2. Centrar: $\mu_{pq} = \sum (x - \bar{x})^p (y - \bar{y})^q \,\mathrm{img}$.
3. Normalizar: $\eta_{pq} = \mu_{pq} / \mu_{00}^{(p+q)/2+1}$.
4. Combinar los η en 7 funciones específicas $h_1, \ldots, h_7$.

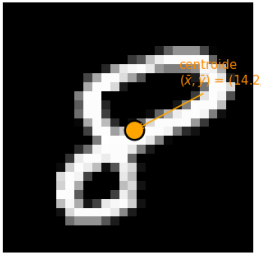
Qué los hace útiles: los h_i son **invariantes a**: Translación (vía centroide), Escala (vía μ_{00}), Rotación (combinaciones específicas).

Lectura técnica: capturan "forma global". Útiles en OCR clásico, menos en MNIST porque los dígitos NO son invariantes a rotación (un 6 rotado 180° es un 9).

Hu — ejemplo visual sobre un 8 (con fórmulas)

Hu moments: 7 fórmulas, cada una con un valor concreto

Imagen (dígito 8)



Las 7 fórmulas + el valor que da en este 8:

$h_1 = \eta_{20} + \eta_{02}$	= +0.353	*[spread (cuán esparcido está)]*
$h_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$	= +1.137	*[elongación (alargamiento)]*
$h_3 = (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2$	= +1.923	*[asimetría direccional]*
$h_4 = (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2$	= +3.118	*[asimetría direccional (otro eje)]*
$h_5 = (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})[\ldots]$	= +6.349	*[asimetría compleja]*
$h_6 = (\eta_{20} - \eta_{02})[\ldots] + 4\eta_{11}[\ldots]$	= -4.061	*[interacción ejes]*
$h_7 = (3\eta_{21} - \eta_{03})[\ldots] - (\eta_{30} - 3\eta_{12})[\ldots]$	= -5.646	*[skew/chiralidad]*

Donde η_{pq} son los \emph{momentos centrales normalizados} (ver slide siguiente).

Cada h_i es una fórmula *distinta* de los η_{pq}. El truco está en la combinación específica: cada h_i resulta invariante a alguna transformación. Los valores son log-transformados para evitar números muy chicos/grandes.

¿Qué son los η_{pq}? — el bloque de construcción

Los h_i se arman a partir de los **momentos centrales normalizados** η_{pq}. Vienen en 3 pasos:

1) Momentos crudos:

$$m_{pq} = \sum_{x,y} x^p \, y^q \, \mathrm{img}(x, y)$$

Ej. m_{00} = masa total. $m_{10}/m_{00} = \bar{x}$ (centroide).

2) Momentos centrales — restando el centroide:

$$\mu_{pq} = \sum_{x,y} (x - \bar{x})^p \, (y - \bar{y})^q \, \mathrm{img}(x, y)$$

Esto los hace *invariantes a translación*.

3) Normalización por escala:

$$\eta_{pq} = \frac{\mu_{pq}}{\mu_{00}^{(p+q)/2+1}}$$

El exponente del denominador hace que η_{pq} sea *invariante a escala*.

¿Qué captura cada h_i? — intuición

Las 7 fórmulas de Hu son *combinaciones específicas* de los η_{pq} pensadas para que cada una sea **invariante a rotación** además de translación y escala.

Hu	Captura	Intuición
$h_1 = \eta_{20} + \eta_{02}$	spread	cuán esparcida está la tinta
$h_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$	elongación	alargado vs. circular
h_3 (cúbica)	asimetría 1	sesgo en una dirección
h_4 (cúbica)	asimetría 2	sesgo en otra dirección
h_5, h_6 (mixtas)	combinaciones más finas	detalles de forma
h_7 (chiralidad)	sentido de rotación	distingue figuras *espejadas*

Por qué importa la invariancia: Hu sobre un 8 rotado 30° o el doble de grande da *los mismos valores*. Útil en OCR clásico pero marginal en MNIST (6 rotado 180° es un 9, no es lo mismo).

Feature 3 — Topológicos (topo, 5)

Idea: features derivadas de la *estructura topológica* de la imagen binarizada.

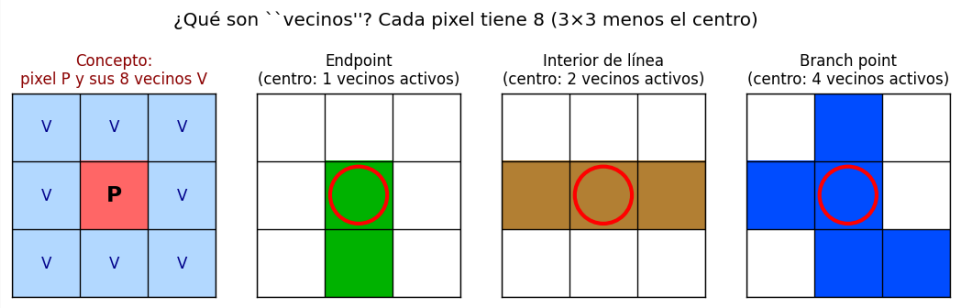
Las 5 features (binarización con threshold 0.3):

1. **Cantidad de huecos**: regiones de fondo NO conectadas al borde. *0/6/9 → 1 hueco; 8 → 2 huecos; 1/2/3/4/5/7 → 0*.
2. **Euler number**: #componentes − #huecos.
3. **Endpoints** en el esqueleto: pixels con 1 vecino (puntas del trazo). *1 tiene 2 endpoints*.
4. **Branch points**: pixels con ≥ 3 vecinos (intersecciones).
5. **Densidad**: fracción de pixels activos.

Por qué es valioso: esta info es *difícil* para un MLP porque requiere análisis no-local. Aporta señal NUEVA que los pixels y proj no proveen.

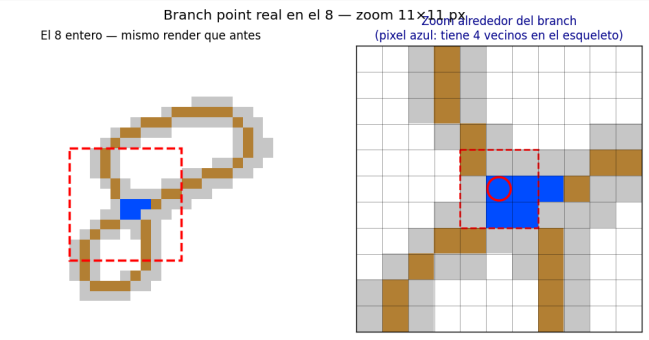
¿Qué es un “vecino”? Concepto + ejemplos

Cada pixel en una imagen 2D tiene hasta **8 vecinos**: los 8 pixels que lo rodean en una ventana 3×3. Después de *esqueletizar* (reducir el trazo a 1 pixel de ancho), contamos cuántos vecinos del esqueleto tiene cada pixel:



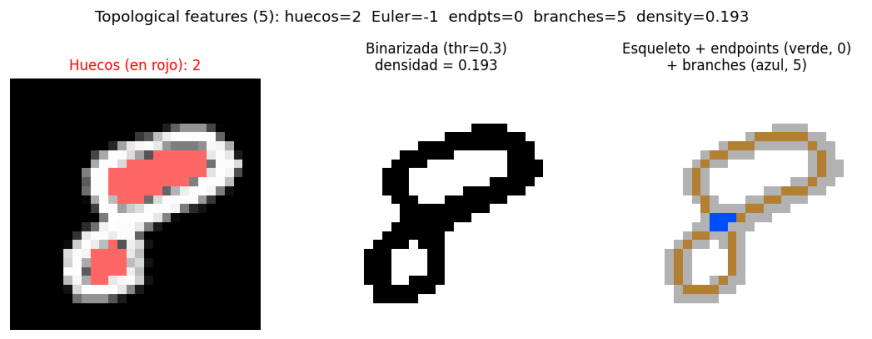
- **1 vecino** = **endpoint**: punta de un trazo.
- **2 vecinos** = interior de una línea (el trazo pasa por ahí).
- **3+ vecinos** = **branch**: una intersección donde varias líneas se juntan.

Branch real en el 8 — zoom para verlo



El pixel azul (centro del círculo rojo) tiene **4 vecinos del esqueleto** en su entorno 3×3 (cuadro punteado rojo): los 4 pixels marrones contiguos. Por eso califica como *branch*. Es el punto donde los dos lóbulos del 8 se conectan.

Topológicos — ejemplo visual sobre un 8



Izquierda: huecos en rojo (este 8 tiene 1 hueco al binarizar; los dos lóbulos del 8 quedaron conectados al binarizar con thr=0.3). Centro: imagen binarizada + valor de densidad. Derecha: el **esqueleto** en marrón, con endpoints en verde y branches en azul.

Las 3 “capas” de la imagen topológica — traducción

Cuando mirás el panel derecho (el del esqueleto), aparecen hasta 4 colores. Acá los traducimos:

Color	Qué es
gris claro	Pixels *binarizados* (la tinta del 8 después de threshold).
marrón	El *esqueleto*: el trazo reducido a 1 pixel de ancho.
azul	*Branch points*: pixels del esqueleto con ≥ 3 vecinos.
verde	*Endpoints*: pixels del esqueleto con 1 vecino.

¿Por qué en este 8 NO hay verde? El 8 no tiene “puntas” —son dos lóbulos cerrados. Por lo tanto el esqueleto no tiene pixels con 1 solo vecino. Compará: un 1 sí tendría 2 endpoints (arriba y abajo); un 7 también.

Feature 4 — HOG (Histogram of Oriented Gradients, 324)

Idea (Dalal & Triggs, 2005): describir la imagen por la *distribución de orientaciones de gradiente* en celdas locales.

Cómo se construye:

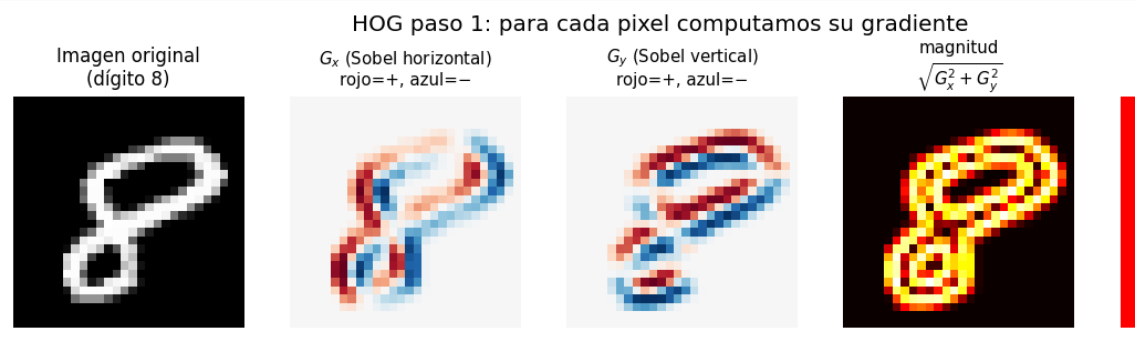
1. Calcular gradientes G_x, G_y en cada pixel (ej. Sobel).
2. Computar magnitud $|G|$ y orientación θ por pixel.
3. Dividir la imagen en *cells* (ej. 7×7 pixels).
4. En cada cell: histograma de orientaciones (9 bins entre 0° y 180°), pesando por la magnitud.
5. Agrupar cells en *blocks* (2×2 cells), normalizar cada block (L2-Hys).

Para 28×28 con cells de 7 px y blocks 2×2: 3×3 blocks $\times$ 4 cells/block $\times$ 9 orient $= 324$ features.

HOG paso 1 — ¿qué es un “gradiente”?

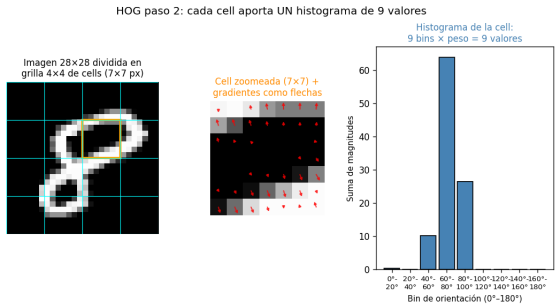
Un **gradiente** mide cuánto cambia el brillo de un pixel al moverse en una dirección. Detecta *bordes*: si dos pixels vecinos son muy distintos, el gradiente ahí es alto.

Para cada pixel calculamos G_x (cambio horizontal) y G_y (cambio vertical) con kernels Sobel: $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * \text{img}$, $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * \text{img}$. De ahí: magnitud $= \sqrt{G_x^2 + G_y^2}$, ángulo $= \arctan(G_y / G_x)$.



HOG paso 2 — de gradientes a histograma por cell

Ahora agrupamos: dividimos la imagen en **cells** de 7×7 pixels (queda una grilla 4×4 de cells). Por cada cell hacemos un *histograma* de las orientaciones que tienen los pixels adentro, ponderando cada voto por la magnitud del gradiente:



Resultado: cada cell aporta **9 valores** (uno por bin de orientación). Total: $4 \times 4 \times 9 = 144$ valores por imagen *antes* de normalizar por block.

HOG paso 3 — normalización por block + total

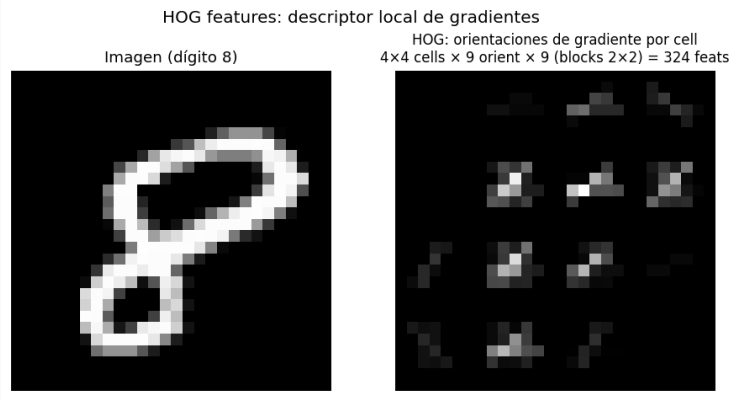
Las celdas se agrupan en **blocks** de 2×2 cells (con *overlap*: cada cell pertenece a varios blocks). El histograma de cada block se **normaliza** (L2-Hys) para que el vector total sea robusto a cambios de iluminación / contraste.

Para 28×28 con cells 7×7 y blocks 2×2:

- Cells totales: $4\times4 = 16$
- Blocks 2×2 con stride 1: $3\times3 = 9$ blocks
- Por block: 2×2 cells $\times$ 9 orient $= 36$ valores
- Total: $9 \times 36 = 324$ features

La intuición clave: HOG codifica "en cada zona local de la imagen, ¿qué orientaciones de borde predominan?". Los dígitos tienen patrones de bordes muy distintivos por zona — un 1 es casi todo vertical en el centro; un 0 es circular; un 7 tiene una horizontal arriba y una diagonal bajando.

HOG — ejemplo visual sobre un 8



Cada celda 7×7 tiene un histograma de orientaciones de gradiente representado como "estrellas" (longitud = peso de esa orientación). Se ve la forma del 8 en los bordes (gradientes fuertes en la silueta).

Data augmentation — generar samples sintéticos

Idea: aplicar transformaciones *conservadoras* a las imágenes del train para crear "copias" que la red trate como samples nuevos.

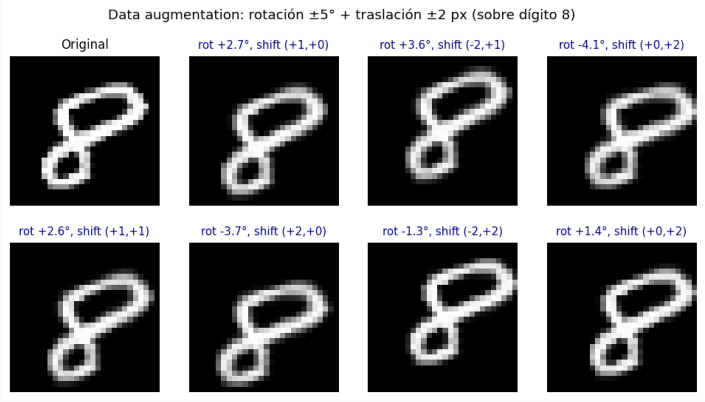
Transformaciones que usamos:

- **Rotación**: ángulo aleatorio en $[-5, +5]$ (`scipy.ndimage.rotate`).
- **Traslación**: shift aleatorio en $\{-2, ..., +2\}$ pixels en cada eje (`scipy.ndimage.shift`).

Para qué sirve: reduce overfitting (la red ve "más datos" efectivos), hace al modelo invariante a esos cambios pequeños, y *especialmente* útil para clases minoritarias: oversampling con augmentation balancea las clases sin duplicar datos idénticos.

En EJ3 lo usamos así: cap @ 1000 + oversample las clases 5 y 8 hasta 1000 cada una usando rotación + traslación aleatorias.

Augmentation — ejemplo visual sobre un 8



Cada imagen es una rotación + traslación aleatoria del mismo 8 original. La red aprende a ser invariante a esos cambios y la clase minoritaria se multiplica sin necesitar más datos reales.

Balance de clases — cap + oversample con aug

Problema: el train EJ3 tiene 3.7× más 1s que 5s. La red aprende "hay muchos 1s, ante la duda voy con 1".

Estrategia que usamos:

1. **Cap**: si una clase tiene más de 1000 samples, reducimos al azar a 1000.
2. **Split** 80/20 estratificado en train/val.
3. **Oversample con augmentation**: las clases 5 y 8 se completan hasta 800 generando *copias* con rotación + traslación aleatoria.

Resultado: dataset balanceado de ∼8.000 samples (800 por clase) en lugar de 12.597 con desbalance original. Val NO se augmenta (evita data leakage).

Noise per-epoch — regularización por perturbación

Idea: en cada época, agregar ruido gaussiano al input antes de entrenar. El ruido se *regenera* cada época, así la red ve un train ligeramente distinto cada vez.

$$X_{\text{epoch}} = X_{\text{train}} + \mathcal{N}(0, \sigma^2), \quad \sigma = 0{,}1$$

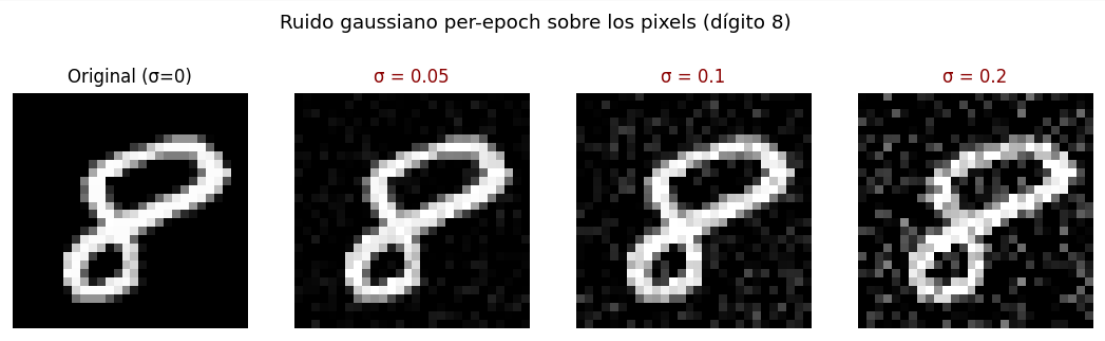
Pero el val/test se evalúa SIEMPRE sobre datos limpios.

```
for epoch in range(max_epochs):
    noise = rng.normal(0, sigma, X_train.shape)
    X_noisy = X_train + noise
    model.train_epoch(X_noisy, Y_train, ...)
    val_acc = evaluate(model, X_val_clean) # <- clean!
```

Por qué funciona: fuerza al modelo a NO memorizar pixels específicos — tiene que aprender la *forma*.

Aporte en EJ3: **+0.48 pts** test (la regularización más efectiva sola).

Noise — ejemplo visual sobre un 8



Con $\sigma = 0{,}1$ (lo que usamos en EJ3) la imagen sigue reconocible pero los pixels individuales ya no son exactos. La red se enfoca en la *forma*, no en valores específicos.

Dropout — qué es, por qué lo probamos, por qué lo descartamos

Idea (Srivastava et al., 2014): durante el training, "apagar" aleatoriamente una fracción p de las neuronas en cada batch.

Inverted dropout (la versión que implementamos):

$$V_j' = V_j \cdot 1[u_j > p]/(1-p), \quad u_j \sim \text{Uniform}(0,1)$$

Aplicado solo en *training*, sólo en *capas ocultas*. En inference: $V_j' = V_j$ (sin máscara).

Por qué funciona en general: cada neurona no puede "confiar" en sus vecinas $\rightarrow$ aprende features útiles por sí sola. Equivale a entrenar un *ensemble* implícito.

En EJ3 NO funcionó:

- Combo + drop 0.5: test=0.9644 ($-$0,28 pts).
- Combo + drop 0.3: test=0.9636 ($-$0,36 pts).

Diagnóstico: el noise per-epoch *ya* regulariza lo mismo que daría dropout. Sumarlos *underfittea* sin agregar info.

Ensemble — promediar varios modelos

Idea: entrenar K versiones del modelo con seeds distintas (cada una explora un mínimo local distinto), y combinar sus predicciones.

Cómo combinar (lo que usamos): $\bar{y} = \frac{1}{K}\sum_k y^{(k)}$, después `argmax`. Alternativa: voting (cada modelo vota `argmax`, se elige la moda).

Por qué funciona:

- Cada seed converge a un mínimo distinto (cada uno con sus sesgos).
- Los errores tienden a ser *independientes* entre modelos.
- El promedio cancela errores correlados negativamente.

En EJ3: con 3 seeds del iter10 (combo + topo + HOG), pasamos de ∼97.5 % (mejor single) a **98.08 %** (+0,5 pts). *Es lo que cerró el gap al 98 %.*

Trayectoria de iteraciones — todo el camino

Iter	Cambio respecto al anterior	test_acc	Δ
0	Baseline (tanh, Adam, L2)	0.9624	—
1	+ balance + ReLU + aug	0.9644	+0.20
2	+ features (proj + Hu)	0.9640	+0.16
3	+ noise per-epoch	0.9672	+0.48
4	ReLU sola (sin combo)	0.9624	+0.00
5	combo (1+2+3)	0.9672	empate
6	combo + dropout 0.5	0.9644	−0,28
7a	combo + dropout 0.3	0.9636	−0,36
7b	combo + drop 0.3, sin noise	0.9652	−0,20
8	cap train_acc=0.99 (early)	0.9511	−1,61
9	combo + topo	0.9692	+0.20
10	combo + topo + HOG	0.9756	+0.64
11	**ensemble 3 seeds del iter10**	**0.9808**	**+0.52**

Total: **+1.84 pts** desde el baseline (96.24 % → 98.08 %).

Lo que más aportó: HOG (+0.64), Ensemble (+0.52), Noise (+0.48).

Lo que NO funcionó: Dropout, ReLU sola, cortar épocas pronto.

El modelo final — anatomía completa

Arquitectura: `[1176, 512, 512, 10]`, ∼870k params.

Input (1176 features):

- 784 pixels crudos
- 56 projection features (row_sums + col_sums)
- 7 Hu moments
- 5 topological features (huecos, Euler, endpts, branches, density)
- 324 HOG features

Activación: ReLU oculta + sigmoide salida. **Init**: He.

Optimizador: Adam lr=5e-4, β_1=0,9, β_2=0,999.

Regularización: L2 ($\lambda = 10^{-4}$) + noise per-epoch ($\sigma = 0{,}1$, solo sobre los 784 pixels) + cap@1000 + oversample con augmentation para 5 y 8.

Training: batch=32, max_ep=500, patience=50.

Ensemble: 3 modelos con seeds 0, 1, 2. Predicción final =

Per-class F1 final (test EJ3)

Clase	F1 baseline EJ3	F1 final EJ3 (ensemble)
0	0.94	**0.986**
1	0.95	**0.991**
2	0.89	**0.971**
3	0.84	**0.984**
4	0.89	**0.988**
5	0.75	**0.982**
6	0.91	**0.983**
7	0.92	**0.977**
8	**0.00**	**0.971**
9	0.92	**0.974**

- El 8 pasó de F1=**0** (imposible) a 0.971 — gracias a `more_digits.csv`.
- El 5 (que era el más débil del baseline EJ3) saltó de 0.75 a 0.982 — gracias al balance + augmentation + HOG.
- Todas las clases con F1 $\geq$ 0.97. Modelo equilibrado.

Conclusiones

Conclusiones EJ2

- **Exp1 (batch $\times$ lr)**: el learning rate óptimo depende del batch size. Online logra la mejor accuracy pero es 10$\times$ más lento. Se elige batch=32 como compromiso eficiencia/rendimiento.
- **Exp2 (optimizadores)**: cada optimizador tiene su rango óptimo de lr. Adam con lr=10^{-3} domina en accuracy y velocidad.
- **Exp3 (arquitecturas)**: heatmap 2D ($n_1 \times n_2$). *Wider domina sobre deeper* en este rango: la primera capa es la más crítica (procesa los 784 píxeles crudos), la segunda aporta poco. Ganador: $[784, 512, 512, 10]$.
- **Exp4 (activación)**: ReLU, tanh y logistic dan resultados aceptables. ReLU es preferida por convergencia más rápida y sin saturación en valores positivos.
- **Exp5 (mejor modelo)**: **[784, 512, 10]** (1 capa oculta) + ReLU + Adam lr=10^{-3}, batch=32. Ganador del heatmap ReLU.
- **Techo de test accuracy $\approx$ 90 %** por la ausencia del dígito 8 en train. El 8 se confunde con 5 y 3; el dígito 5 (271 muestras) también sufre.

Referencias al material

- Clase 9 — Optimización (GD, SGD, Momentum, Adam)
- Clase 11 — Perceptrón Multicapa (forward pass, T. Aprox. Universal)
- Clase 12 — Métricas y Sobreajuste (accuracy, F1, confusion matrix)
- Clase 13 — Backprop (regla de la cadena, δ por capa)
- Clase 13 — Regularización (early stopping, L2 weight decay)